



SMART HOME PROJECT

Team Project :-



- Kareema Esmail Mohammed
- Fatma AbdelAziz Lashin
- Noha Sherif Helmy
- Heba Magdy Mohammed
- Mariam Samy Abdelrahman

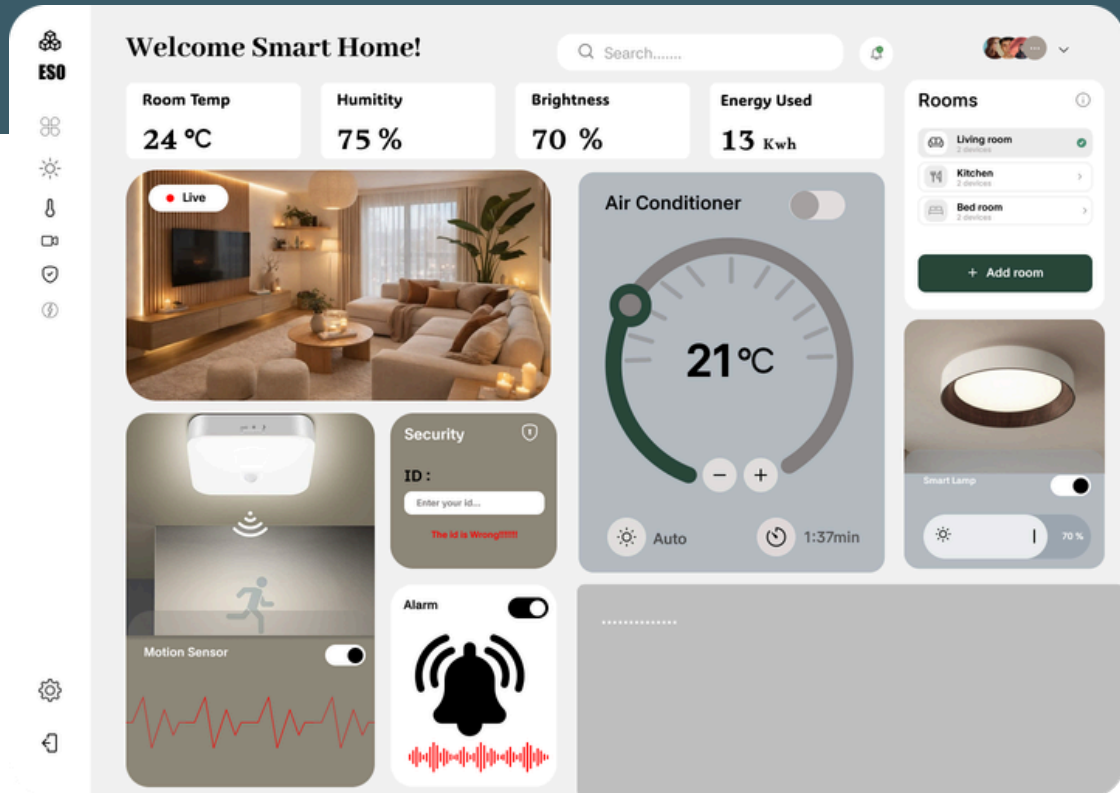
**UNDER THE SUPERVISION OF :
DR / SARA**



Introduction

- ▶ In the modern era of automation, the Internet of Things (IoT) and embedded systems have revolutionized how we interact with our living spaces. Smart homes are no longer a futuristic concept but a present reality, requiring robust, scalable, and intuitive software solutions to manage an ecosystem of heterogeneous devices.
- ▶ This project presents the design and implementation of a Smart Home Control Panel, an advanced desktop dashboard engineered using JavaFX and core Object-Oriented Programming (OOP) principles, specifically Inheritance and polymorphism.
- ▶ The system simulates a real-time smart home environment by unifying diverse device categories—such as Lighting (SmartBulbs, RGBStrips), Climate Control (Thermostats, AirPurifiers), and Security Systems (Cameras, MotionSensors)—under a single, cohesive abstract framework.
- ▶ The system simulates a real-time smart home environment by unifying diverse device categories—such as Lighting (SmartBulbs, RGBStrips), Climate Control (Thermostats, AirPurifiers), and Security Systems (Cameras, MotionSensors)—under a single, cohesive abstract framework.
- ▶ By leveraging JavaFX Properties for dynamic, real-time UI updates and incorporating design patterns like the Observer pattern, this application ensures seamless state monitoring and interactive device control. This report details the architectural class hierarchy, the integrated JavaFX graphical components, and the technical implementation that satisfies the demands of modern embedded-systems coursework.

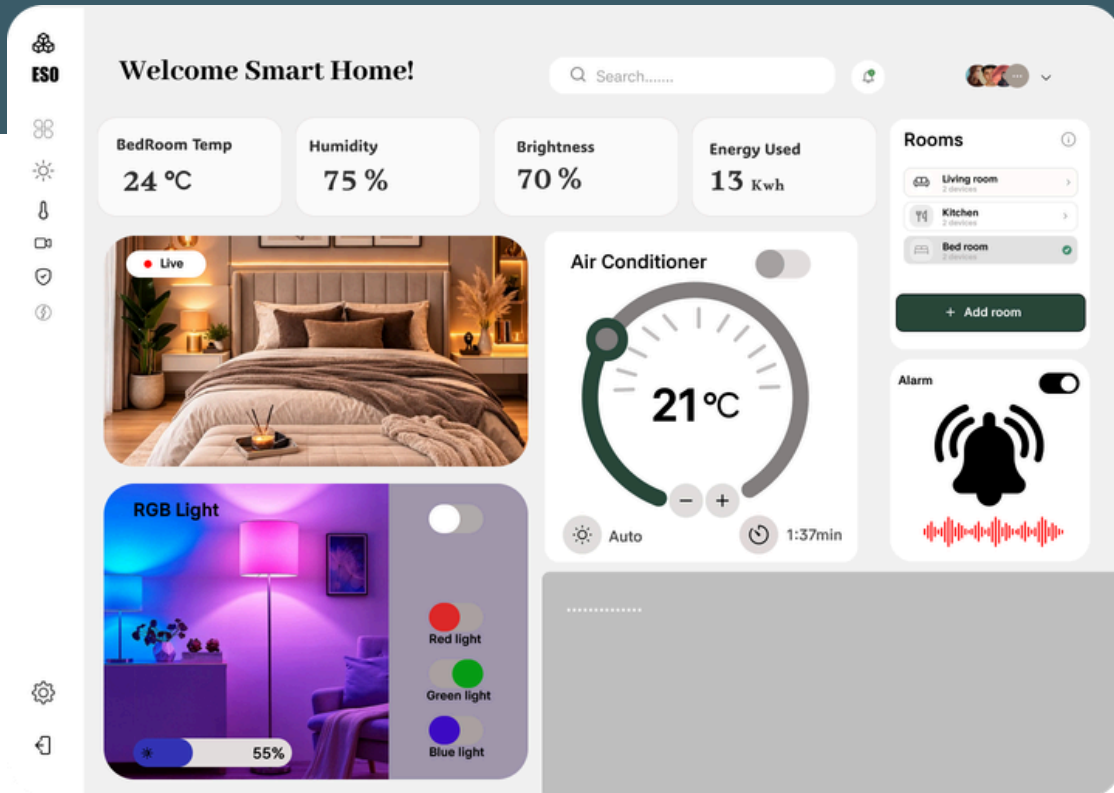




LIVINGROOM VIEW CONTROL PANEL

- The Living Room interface features a live camera feed preview, a centralized arc-shaped Air Conditioner controller, a Smart Lamp with a brightness dimmer slider, a Motion Sensor tracker with a simulated pulse wave, an ID-based Security access panel, and a responsive Safety Alarm system

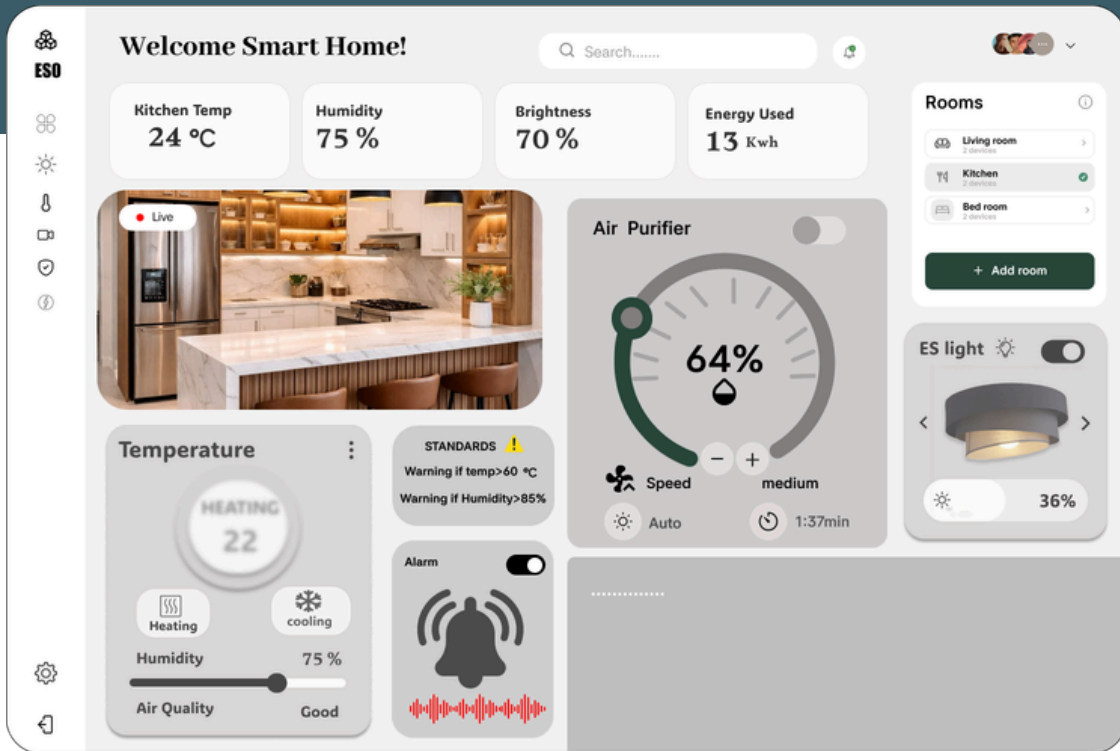




BEDROOM VIEW CONTROL PANEL

- The Bed Room interface features a live camera feed preview, a centralized arc-shaped Air Conditioner controller, a specialized RGB Light control panel with individual channel toggles (Red, Green, Blue) and a dimming slider, and a responsive Safety Alarm system.



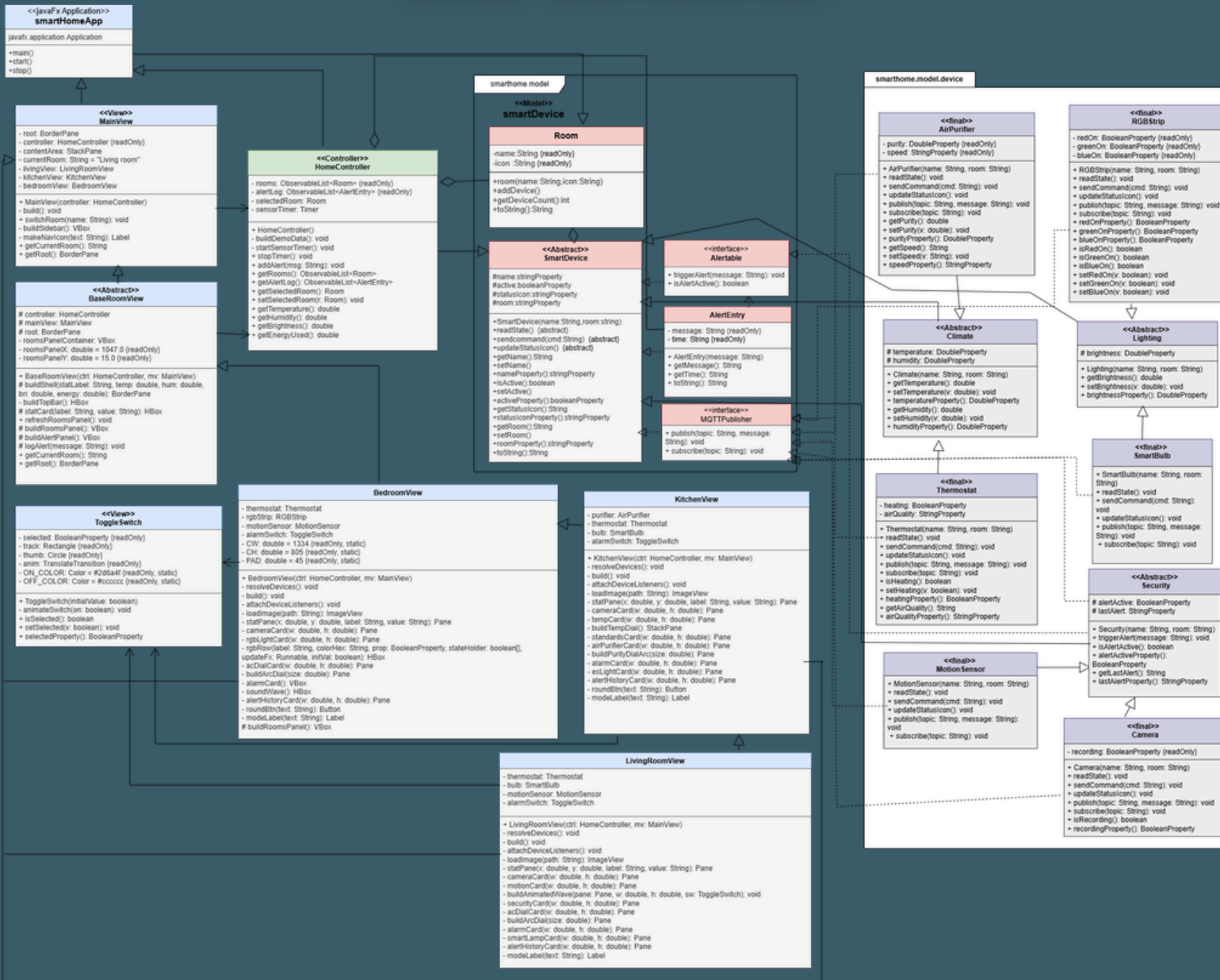


KITCHEN VIEW CONTROL PANEL

- The Kitchen interface features a live camera feed preview, a circular Air Purifier gauge with speed settings, a dedicated Climate Panel with heating/cooling toggles and a humidity slider, an environmental Standards warning threshold card, an ES Light controller, and a synchronized Safety Alarm system.



UML DIAGRAM



System Architecture

The structural architecture of the Smart Home System is designed following the **Model-View-Controller (MVC)** architectural pattern to ensure high modularity, scalability, and loose coupling between the system components.

Main Application Entry and View Layer

The user interface architecture forms a hierarchical structure managed by JavaFX components:

- SmartHomeApp**: Inherits from `javafx.application.Application` and serves as the main entry point of the entire application, responsible for launching the primary life cycle (**main**, **start**, **stop**).



- **MainView**: Acts as the primary root container (**BorderPane**). It maintains a strict composition relationship with the specific room views (**LivingRoomView**, **KitchenView**, **BedroomView**) and a direct association with the **HomeController**.
- **BaseRoomView**: An abstract base class that implements common user interface setups for individual rooms. The specific views (**LivingRoomView**, **KitchenView**, **BedroomView**) inherit (extend) from this abstract class, ensuring UI consistency across the dashboard.
- **ToggleSwitch**: A custom UI component that is heavily used by the views via aggregation to toggle the state of various smart devices.

The Controller

- **HomeController**: This class acts as the central brain or mediator of the application. It holds a direct bidirectional association with the **MainView**. It receives user interactions from the View layer and coordinates data updates with the Model layer. It maintains an observable list of rooms and manages a system timer for periodic tasks.

The Model

The system represents physical spaces and devices using a rich object-oriented domain model:

- **Room**: Encapsulates the concept of a physical space. It contains a collection of smart devices managed through a structured aggregation relationship.
- **SmartDevice (Abstract Class)**: Forms the backbone of the hardware abstraction layer. It defines abstract operations like **readState()** and **sendCommand()**. All specialized smart hardware components inherit from this base class.

Specialized Devices Sub-packages (**smarthome.model.device**):

- **Climate (Abstract)**: Extended by the concrete **Thermostat** and **AirPurifier** classes to handle temperature and air purity.
- **Lighting (Abstract)**: Extended by **SmartBulb** and **RGBStrip** to manage brightness and color states.
- **Security (Abstract)**: Extended by **Camera** and **MotionSensor** to handle monitoring and intruder alerts.

Communication

To allow independent devices to communicate status changes seamlessly, the system integrates messaging interfaces:

- **Alertable**: Devices that can trigger emergencies (like the **MotionSensor**) implement the **Alertable** interface, creating **AlertEntry** objects that are communicated back to the **HomeController** to update the global notifications pool.
- **MQTTPublisher**: The system implements a decoupled messaging behavior where security, climate, and lighting devices implement or associate with a publication interface (**publish**, **subscribe**), simulating real-time IoT network behavior.



→ SMART HOME MODEL CLASSES

SmartDevice Class (Abstract Base Class):

The SmartDevice class serves as the abstract foundation of the system architecture, defining common attributes and behaviors for all smart devices using key OOP principles.

Key Components:

- **JavaFX Properties:** Uses `StringProperty` and `BooleanProperty` for name, room, active, and statusIcon. This enables Data Binding, ensuring device state changes instantly update the UI in real time.
- **Abstract Methods:** Enforces essential behaviors that every subclass (e.g., `SmartBulb`, `Thermostat`) must uniquely implement :
 - `readState()`: Simulates reading real-time sensor updates.
 - `sendCommand(String cmd)`: Handles control actions from the dashboard.
 - `updateStatusIcon()`: Dynamically updates the device's visual icon based on its state.
- **Design Note:** Subclass fields must fully initialize before executing icon logic, so `updateStatusIcon()` is intentionally called within subclass constructors rather than the super constructor.

Room Class :

The Room class acts as a container that groups smart devices based on their physical location (e.g., Living Room, Kitchen).

- **Attributes:** Holds basic information for each room, including its name and a path to its visual icon.
- **ObservableList (devices):** Instead of a standard `ArrayList`, it utilizes `JavaFX ObservableList`.
- This automatically triggers UI layout updates whenever a device is added or removed from the room.
- **Methods:** Provides helper functions to easily add devices (`addDevice()`) and retrieve the total number of connected devices (`getDeviceCount()`).

MQTTPublisher Interface :

The MQTTPublisher interface establishes a standardized contract for simulating network communication within the smart home setup.

- **Purpose:** It mimics the MQTT protocol, which is the industry standard lightweight messaging protocol for IoT and embedded systems.
- **Core Functions:** `publish(String topic, String message)`: Simulates sending device data or state updates to a central broker.
- `subscribe(String topic)`: Simulates listening for incoming remote commands or automation triggers.



Alert Entry Class :

- This data model class represents a single log entry in the system's security and safety alert history. It automatically captures the precise timestamp when an alert occurs using Java's `LocalTime API`, formatting it into `HH:mm:ss`. It encapsulates the alert message and time, overriding the `toString() method` to provide a unified, clean string representation for UI list views

Alertable Interface :

- This interface defines the core behavioral contract for any smart device or system module capable of handling security emergencies or environmental hazards. By enforcing the implementation of `triggerAlert()` and `isAlertActive()`, it decouples the alert-triggering logic from specific device implementations, ensuring high extensibility across different types of hardware sensors in the smart home grid.
-

➔ HOME CONTROLLER CLASS

- This central coordinator class implements the Controller logic of the MVC architecture. It manages global state across the entire automation network using `ObservableList` collections for reactive UI updates. It acts as an orchestrator by generating simulation data (`buildDemoData()`) and running a background scheduling engine (`TimerTask`) that polls physical hardware state through standard hardware polling cycles (`readState()`). Furthermore, it computes real-time ambient mathematical averages across dynamic device streams for temperature, humidity, and lighting levels.
-

➔ SMART HOME APP CLASS

- This main application class serves as the primary deployment entry point for the system, extending JavaFX's base `Application` lifecycle. It initializes the central core architectural layout by instantiating the backend orchestrator controller (`HomeController`) and passing it directly into the primary UI frame (`MainView`). It configures the window's foundational runtime container (`Scene`), dynamically attaches the global structural stylesheet (`style.css`), sets window dimensional constraints, and guarantees secure hardware lifecycle management by cleanly stopping background timer tasks (`stopTimer()`) during system shutdown sequences.



SMART HOME MODEL DEVICES

AirPurifier Class :

- The AirPurifier class represents a smart air-purifying device in the home automation system.
- It extends the Climate device model and implements the MQTTPublisher interface to support IoT communication.

The purifier maintains two main properties:

- **purity (DoubleProperty)** : shows the current air quality level.
- **speed (StringProperty)** : represents the fan speed (low, medium, high).
- When the device is active, the **readState()** method updates the air purity with a simulated value to reflect real-time changes.
- The **sendCommand()** function handles on/off control and speed changes, then publishes the command through MQTT using a topic based on the purifier's name.
- The class also updates its status icon depending on whether the purifier is running.
- It includes full getter/setter methods and publishes MQTT messages to integrate with the smart-home system.

Camera Class :

- The Camera class represents a smart security camera in the home automation system.
- It extends the Security device model and implements the MQTTPublisher interface, allowing it to communicate through MQTT.

The camera includes one main property:

- **recording (BooleanProperty)** :- indicates whether the camera is currently recording.
- The constructor initializes the camera as active by default.
- The **sendCommand()** method handles all control actions: turning the camera on/off and starting or stopping recording. Each command is also published via MQTT using a topic based on the camera's name.
- The **updateStatusIcon()** method switches the icon between an active camera (📹) and an inactive state (🚫).
- Like other devices, the class provides getter methods and integrates easily with the smart-home system through MQTT publishing.

Climate Class :

- The Climate class is an abstract base class used for all smart devices that measure environmental conditions such as temperature and humidity.

It extends the general SmartDevice class and adds two reactive properties:



- **temperature (DoubleProperty) :-** stores the current temperature value (default 22°C).
- **humidity (DoubleProperty) :-** stores the current humidity level (default 75%).
- These properties use JavaFX bindings, allowing the UI to automatically update whenever their values change.
- The class provides getter, setter, and property-accessor methods for both fields, so any subclass (such as an Air Purifier or Thermostat) can read, modify, and bind these values within the smart-home system.

Lighting (Abstract Class)

- This abstract architectural class serves as the foundational blueprint for all illumination-based hardware components within the automated network, directly extending the core generic **SmartDevice model**. It establishes common physical properties by encapsulating a shared **DoubleProperty** parameter for managing light luminosity thresholds (**brightness**), initialized at a baseline of 70%. By providing concrete getter, setter, and property binding hooks for this brightness variable, it ensures unified behavioral patterns and standardizes the API layout for all specialized child lighting devices (**such as bulbs and LED strips**) across different room modules.

RGB Strip Class :

- This final hardware subclass models an advanced entertainment color-mixing light fixture, extending the functional **Lighting** base class and implementing the **MQTTPublisher** interface. It introduces multi-channel encapsulation by managing three independent **BooleanProperty** streams for discrete color states (Red, Green, and Blue), allowing synchronized bidirectional UI data binding. Its hardware polling cycle (**readState()**) conditionally simulates dynamic current spikes or user dimming behaviors using a custom random bound profile (**30% to 100%**) strictly when the unit is active, otherwise flattening the output to 0.0%. It overrides system state visualizations between a multi-color spectrum icon (🌈) and a dark power-down state (■) while maintaining proper physical logging through network telemetry command mapping.

Motion Sensor Class :

- This final device class implements the peripheral sensing logic for security zones, extending the structural Security base class and implementing the **MQTTPublisher interface** for future IoT network bridging. It leverages encapsulation to handle real-time hazard simulation inside its hardware polling loop (**readState()**), utilizing a random uniform distribution profile to autonomously fire reactive alerts via constructor inherited callbacks (**triggerAlert()**).



Furthermore, it overrides behavior dynamically by managing machine-state icons depending on whether the hardware is actively tracking (👁️), idling (😴), or experiencing an ongoing security breach (🚨), while transparently outputting runtime command payloads over

Security Class (Abstract Intermediate Class) :

- The Security class serves as an intermediate abstract class that extends SmartDevice and implements the Alertable interface, acting as the specialized base for all home security hardware like cameras and motion sensors.
- It introduces security-specific JavaFX properties including alertActive to track if a security breach has occurred and lastAlert to store the description or timestamp of the most recent security event. By implementing the Alertable interface, it defines the core logic for triggerAlert, which instantly flips the device into an active alert state and updates the log message, automatically forcing the UI dashboard to highlight the warning in real time through JavaFX properties.

SmartBulb Class :

- The SmartBulb class is a concrete subclass that extends Lighting and implements the MQTTPublisher interface, representing a standard controllable light source. Its constructor explicitly calls updateStatusIcon to set the initial visual state, choosing a lightbulb emoji when active and a flashlight emoji when inactive.
- The class overrides readState to simulate dynamic hardware telemetry by generating a random brightness value between 30 and 100 percent. Control inputs are processed via the sendCommand method, which decodes standard "ON", "OFF", or specific "BRIGHTNESS" commands to mutate device states before triggering a network update via the simulated MQTT publish method to the specific hardware topic path.

Thermostat Class :

- The Thermostat class is a concrete hardware implementation extending Climate and implementing MQTTPublisher to simulate a smart temperature management unit.
- It introduces dedicated tracking properties including heating to toggle environmental modes and airQuality to hold localized environmental safety status strings. Its readState method generates realistic thermodynamic fluctuations around a base temperature of 22 degrees Celsius during heating or 18 degrees Celsius during cooling alongside synchronized random shifts in indoor humidity and ambient air cleanliness levels. The sendCommand method listens for comprehensive state modifications like mode flipping or incremental temperature adjustments, which dynamically alters the status icon between fire, snowflake, or neutral thermometer graphics before broadcasting the adjustment to the central telemetry broker.



SMART HOME VIEWS

➔ Base Room View :

- The BaseRoomView class is an abstract parent class responsible for building the shared UI structure for all room screens in the Smart Home system.
 - It provides the layout foundation and reusable components so that each room (Living Room, Bedroom, Kitchen, etc.) can focus only on its own devices and design.
 - The class builds a consistent top navigation bar, a scrollable main canvas, and a rooms navigation panel placed at fixed coordinates to allow switching between rooms.
 - It also provides helper methods for creating small statistic cards, alert history panels, and for logging alert messages with timestamps.
 - The BaseRoomView ensures that all room views have a unified appearance, synchronized navigation, and the same interactive behavior, while still allowing each room to place its own custom UI in the center area.
-

➔ Main View :

- The MainView class represents the main user interface container for the entire Smart Home application.
- It is responsible for building the overall layout structure, managing room navigation, and switching between the different room views.
- The class initializes the sidebar, the central content area, and creates cached instances of all room views (Living Room, Kitchen, and Bedroom).
- When the user selects a room, the `switchRoom()` method updates the center of the UI with the corresponding view, updates the selected room in the controller, and refreshes the navigation panel in all room views to keep the highlighting consistent.
- The sidebar includes the application logo, navigation icons, and basic control buttons.
- By managing layout, navigation, and room synchronization, the MainView acts as the core container that connects the GUI with the controller and ensures a smooth and unified user experience across the whole system.



➡ Toggle Switch :

- The `ToggleSwitch` class implements a custom on/off switch UI component used across the Smart Home interface.
 - It consists of a background track and a circular thumb that slides smoothly between two positions using a `TranslateTransition`.
 - The switch maintains a `BooleanProperty` called `selected`, which represents its current state and allows the UI or devices to react to changes.
 - When the user clicks the switch, the component toggles its state, animates the thumb movement, and updates the track color (green for ON, gray for OFF).
 - This class provides a simple, reusable, and visually appealing toggle control that behaves consistently across all room views.
-

➡ LivingRoomView :

- The `LivingRoomView` class is the main graphical interface for the Living Room module in the Smart Home application.
- It extends `BaseRoomView`, meaning it inherits the common layout structure (top bar, rooms panel, etc.) and then adds all the Living-Room-specific components such as the AC dial, camera feed, motion sensor, and the interactive smart-lamp card.
- This class is responsible for constructing the full UI, binding device states to visual components, and reacting to device updates (temperature, brightness, motion, etc.) in real time.

2. Device Resolution

- At construction time, the class automatically locates the devices that belong to the Living Room:

```
private Thermostat thermostat;  
private SmartBulb bulb;  
private MotionSensor motionSensor;
```

- These are fetched from the first room in the controller.
- This mechanism allows the view to stay synchronized with the underlying data model and ensures every UI card directly controls the correct device.

2. Building the Layout

- The `build()` method constructs the entire interface:

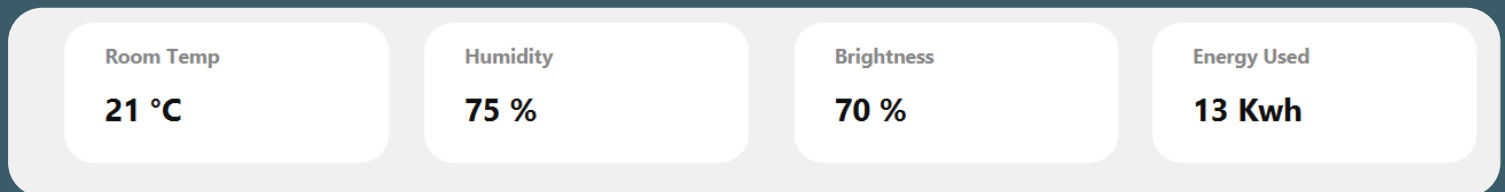


Stat Cards :

Four rectangular “stat cards” show:

- Room temperature
- Humidity
- Brightness
- Energy usage

These cards are placed at absolute coordinates to match a fixed dashboard-style design.



3. Middle Section Components

Camera Feed

- A full-width camera card displays a live snapshot of the room.

It includes:

- Rounded-corner image clipping
- "Live" badge with a red indicator
- Graceful fallback if the image is missing

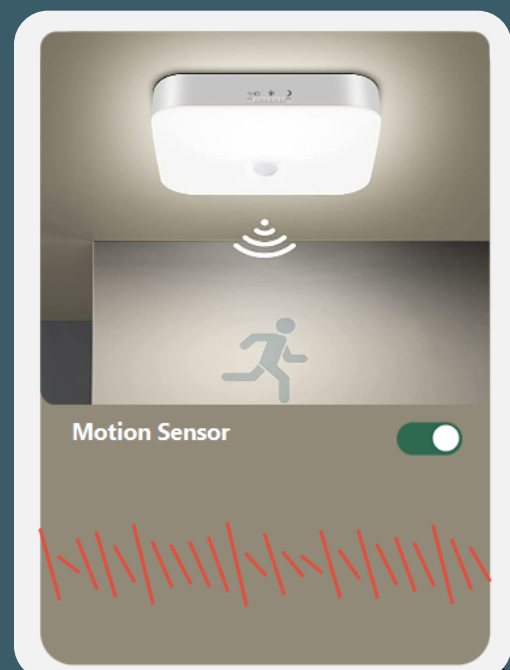


Motion Sensor Card

- The animation is done using a Timeline that continuously changes the Y-coordinates of wave lines

This pane shows:

- A sensor image
- A toggle switch bound to the motion sensor's active state
- A red dynamic wave animation that activates only when the sensor is ON



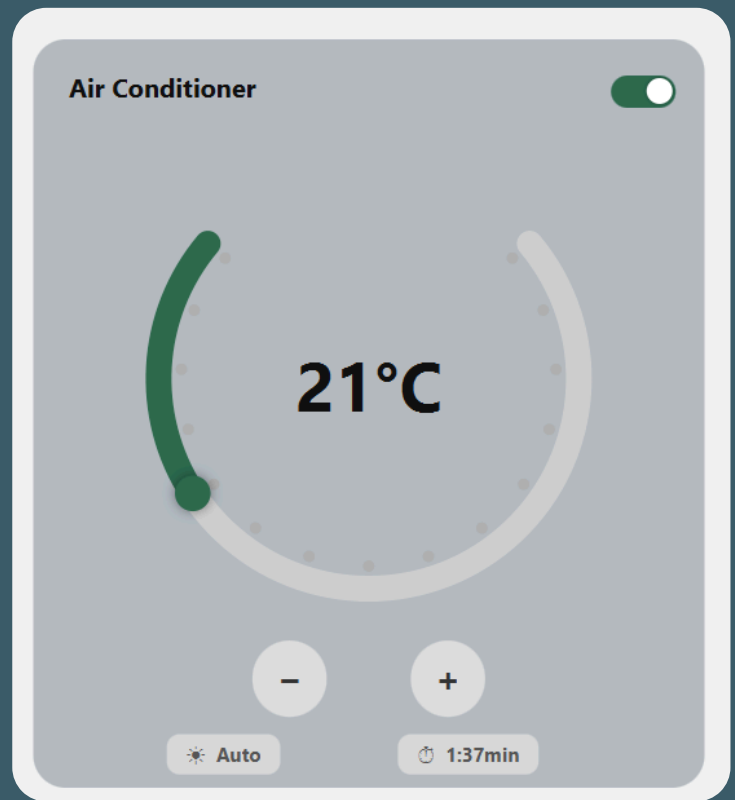
Air Conditioner Dial

- This is one of the most complex components in the view.

It includes:

- A toggle switch bound to the AC's active state
- A circular temperature arc dial
- A draggable ball whose position is not user-controlled (temperature comes only from model)
- Plus and minus buttons to increase/decrease temperature through device commands
- Auto/Timer mode labels

The arc automatically updates visually when the thermostat's temperature changes. Holding the mouse freezes the animation based on your project requirement.

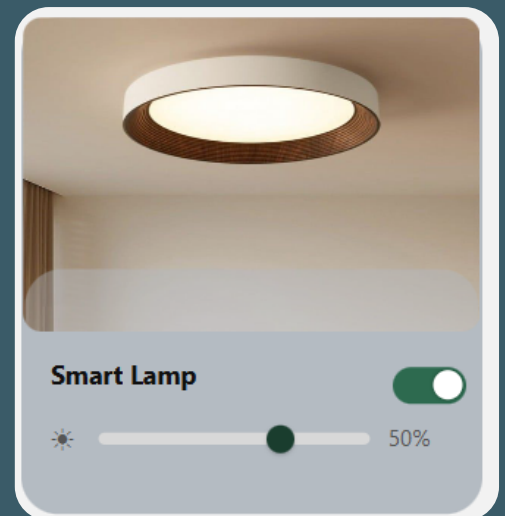


4. Smart Lamp Card

The Smart Lamp card includes:

- Image preview
- Toggle switch bound to the SmartBulb model
- Slider bound to brightness
- Smooth brightness-based color adjustment using a **ColorAdjust** effect
- Saved brightness value when turning the lamp OFF

The brightness logic is identical to the ES Light system (as requested).



5. Alarm System

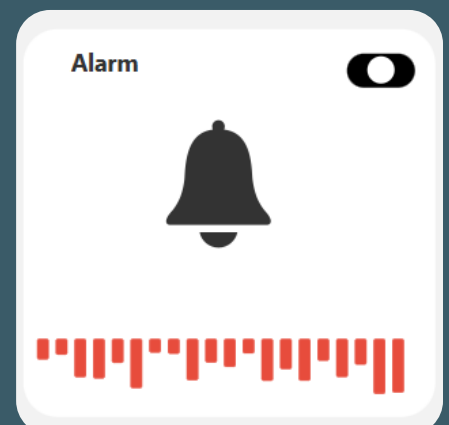
The alarm card includes:

- Toggle switch
- Bell icon (or image)
- Animated sound bars
- Bell shake animation

This is orchestrated inside `attachDeviceListeners()`, which centralizes all event listeners.

The alarm is also automatically triggered when:

- Temperature exceeds a threshold
- Motion is detected
- The AC is turned on
- The Smart Lamp is turned on



6. Alert History

- A scrollable list at the bottom shows all logged alerts with timestamps.
- It is fully synchronized with the controller's global alert log.



7. Device Listeners

All dynamic behavior is handled here:

- Temperature changes
- Humidity changes
- Motion activity
- AC activation
- Lamp activation

This provides a real-time responsive room view.

➔ KitchenView :

- The KitchenView class is the main graphical interface for the Kitchen module in the Smart Home application.
- It extends BaseRoomView, meaning it inherits the common layout structure (top bar, rooms panel, etc.) and then adds all the Kitchen-specific components such as the Air Purifier dial, camera feed, standards card, alarm view, and the interactive ES Light card.
- This class is responsible for constructing the full UI, binding device states to visual components, and reacting to device updates (temperature, humidity, air purity, etc.) in real time.

1. Device Resolution

- At construction time, the class automatically locates the devices that belong to the Kitchen (fetched from room index 1 in the controller):

```
private AirPurifier purifier;  
private Thermostat thermostat;  
private SmartBulb bulb;
```

- This mechanism allows the view to stay synchronized with the underlying data model and ensures every UI card directly controls the correct device.



2. Building the Layout

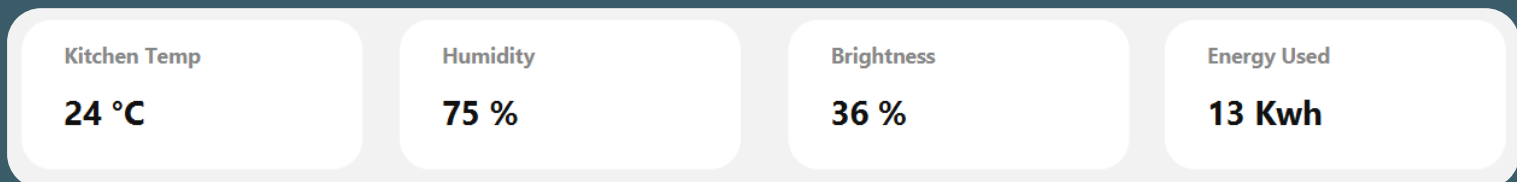
- The `build() method` constructs the entire interface dynamically

Stat Cards :

Four rectangular “stat cards” show:

- Room temperature
- Humidity
- Brightness
- Energy usage

These cards are placed at absolute coordinates to match a fixed dashboard-style design.



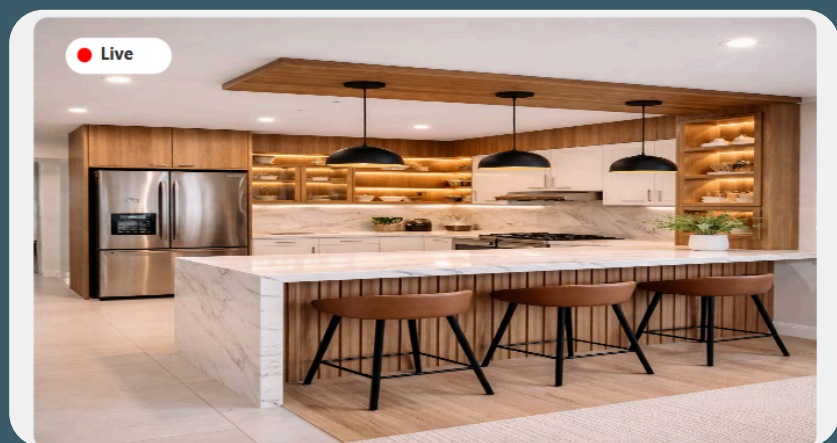
3. Middle Section Components

Camera Feed

- A full-width camera card displays a live snapshot of the kitchen.

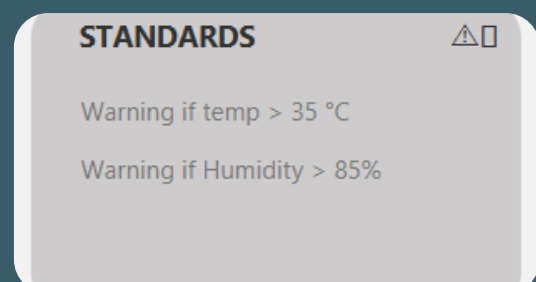
It includes:

- Rounded-corner image clipping
- "Live" badge with a red indicator
- Graceful fallback if the image is missing



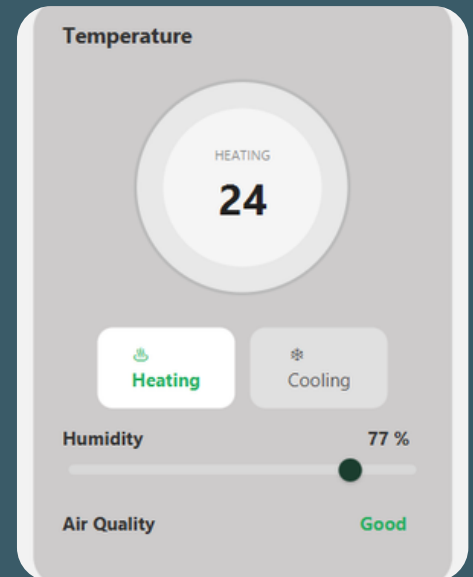
Standards Card (Thresholds)

- A warning display panel that lists the system safety limits:
- Warning if temp > 35 °C
- Warning if Humidity > 85



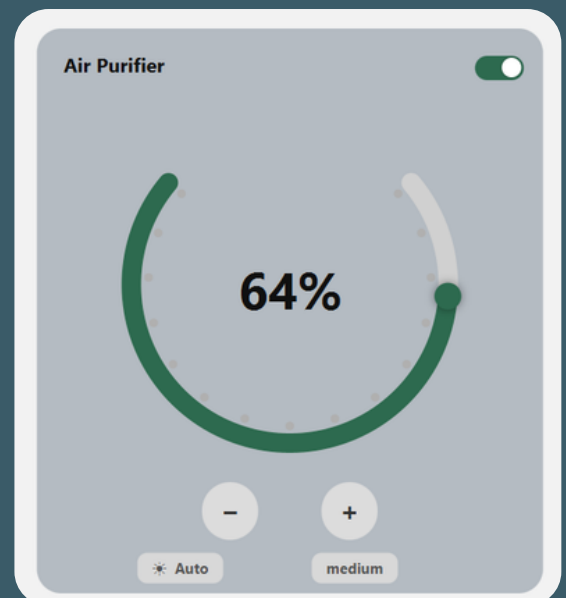
Temperature Card

- A comprehensive environmental controller containing:
- A circular compact dial that supports dragging or scrolling to update temperature values.
- Heating and Cooling mode buttons with active/normal style toggling.
- A Humidity slider bi-directionally bound to the thermostat model.
- An Air Quality label indicator (Good / Moderate / Poor) that changes color dynamically.



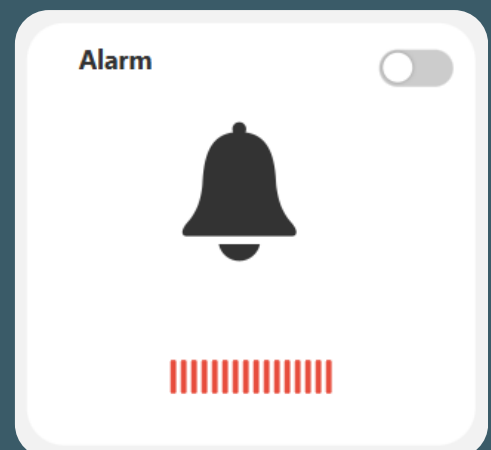
Air Purifier Card

- Replaces the traditional AC slot with a custom circular arc dial showing the air purity percentage.
- Features integrated "+" and "-" round buttons to step-control the purity levels, an Auto-mode indicator, a speed level text property binding, and smooth opacity fading when disabled via its main toggle switch.



Alarm Card

- Features a large bell icon combined with an active HBox soundwave bar animation (built using 16 independent ScaleTransition bars).
- The animations trigger automatically when the alarm is turned ON and reset completely when turned OFF.



ES Light Card

- Formats the Smart Lamp slot with a custom image overlay that responds to brightness changes using a ColorAdjust effect.
- Includes smart functionality to save the current brightness level before turning off, and restores that exact value upon turning the switch back on..



Alert History Card

- A dedicated logging panel aligned perfectly with the alarm row.
- Contains a scrollable ListView bound to the controller's main alert log, highlighting active system warnings dynamically in a distinct dark red color.



Automation & Device Listeners

- The `attachDeviceListeners()` method registers property listeners to create an intelligent reactive ecosystem:
- High **Temperature / Humidity Alert**: If the temperature exceeds 35°C or humidity exceeds 85%, the system automatically toggles the safety alarm switch and logs an alert.
- Auto-Purification: When humidity breaks the 85% threshold, the Air Purifier automatically switches on (`purifier.setActive(true)`) to stabilize the room environment.
- Security Logs: Any device state change (**turning a device ON/OFF or shifting values**) instantly logs a new `AlertEntry` into the global history tracking.



➡ BedRoom View :

- The BedroomView class is the main graphical interface for the Bedroom module in the Smart Home application.
- It extends BaseRoomView, inheriting the foundational dashboard structure (top stats, shared dynamic side rooms panel, etc.) and implements customized layouts tailored for a bedroom environment, including an RGB dynamic strip controller, a traditional AC dial, an enhanced motion-responsive safety toggle, and an alarm system.
- This class manages absolute layout assembly, bi-directional property binding with device models, and real-time interface adjustments based on environmental changes (temperature, humidity, and active motion detection).

1. Device Resolution

- At construction time, the class automatically extracts and caches specific hardware devices belonging to the Bedroom module (fetched from room index 2 in the controller's shared room architecture):

```
private Thermostat thermostat;  
private RGBStrip rgbStrip;  
private MotionSensor motionSensor;
```

- This lookup step establishes target synchronization between the reactive JavaFX properties and backend data changes, ensuring user inputs immediately affect the targeted physical device.

2. Building the Layout

- The `build() method` initializes and updates the shell settings, then constructs the structural canvas layer.

Stat Cards :

Four rectangular “stat cards” show:

- Room temperature
- Humidity
- Brightness
- Energy usage

These cards are placed at absolute coordinates to match a fixed dashboard-style design.



BedRoom Temp

21 °C

Humidity

75 %

Brightness

55 %

Energy Used

13 Kwh

3. Middle Section Components

Camera Feed

- A full-width camera card displays a live snapshot of the kitchen.

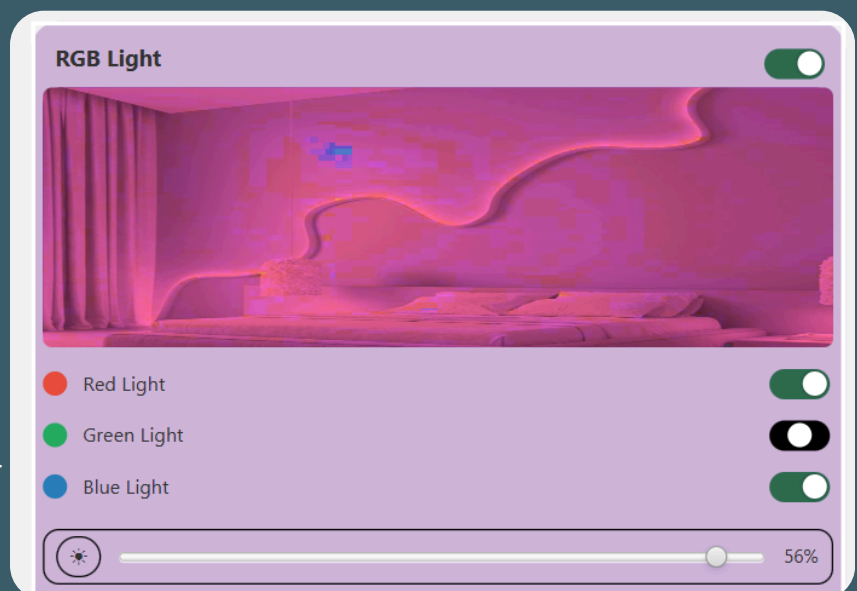
It includes:

- Rounded-corner image clipping
- "Live" badge with a red indicator
- Graceful fallback if the image is missing



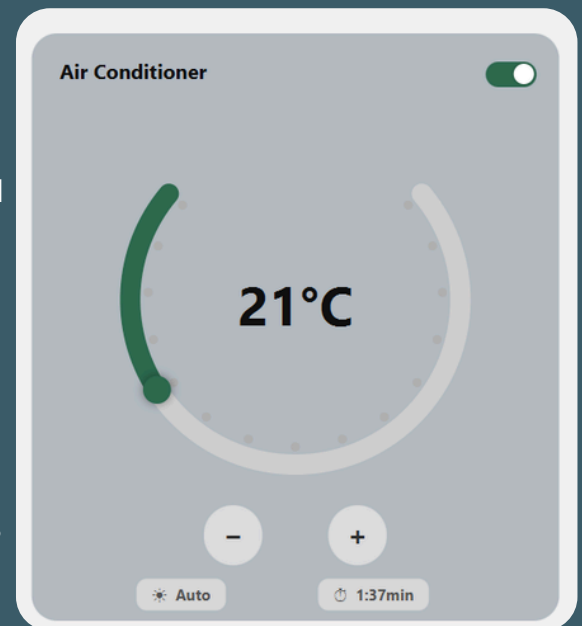
RGB Light Card

- A versatile ambient light controller pane featuring layered mixing mechanics:
- A color preview box driven by a gradient background overlay and a ColorAdjust filter effect that shifts hue based on channel math.
- Three individual color channel toggles (Red, Green, Blue) that can be mixed together to recalculate real-time hex colors dynamically.
- An absolute Brightness slider bi-directionally bound to the strip model that deactivates safely and tracks state configurations when the main toggle switch is cut off



Air Conditioner Card

- A comprehensive cooling interface module that occupies the center slot:
- An arc-based temperature dial featuring decorative ticks, a centered text display, and a floating coordinate knob tracker.
- Step adjustment "+" and "-" round buttons mapped to system temperature step commands (TEMP_UP / TEMP_DN).
- Status badges representing active background routines (Auto climate adjustment and timer counts).
- Auto-fading behavior that dims and disables active sub-controls if the primary AC toggle is turned off.



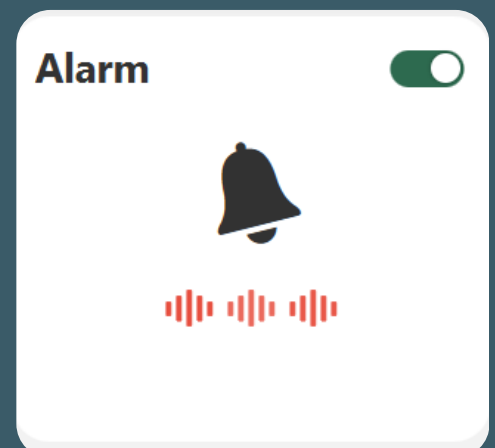
Alert History Card

- Placed at the bottom-right corner to map system activity chronologically.
- Utilizes a transparent-styled JavaFX ListView holding global AlertEntry items, styled with custom cell factories to print warning logs in a distinctive dark red tone.



Alarm Card

- Features a large bell icon combined with an active HBox soundwave bar animation (built using 16 independent ScaleTransition bars).
- The animations trigger automatically when the alarm is turned ON and reset completely when turned OFF.



Automation & Device Listeners

- The `attachDeviceListeners()` method attaches structural property observers to automate security and environmental routines within the room:
- Overheat Prevention Alarm: Monitors the thermostat's temperature; if it spikes beyond 35°C, the code updates the emergency alarm toggle state to true using `Platform.runLater()` and appends a critical system warning log.
- Device Boot Alerts: Observes power property transitions across the AC units and RGB light circuits to instantly notify the global event logging stack upon boot.
- **Motion-Triggered Safety:** Tracks active boolean changes from the `MotionSensor`. If movement is caught while the room is armed, it forces an automated security response, tripping the main alarm switch to active

TOOLS

- The Smart Home system was developed using **JavaFX**, which serves as the main framework for building the graphical user interface.
- JavaFX provides a rich set of UI components such as Labels, Buttons, ListView, TextField, and layout managers including HBox, VBox, BorderPane, and StackPane, which allow structuring the interface in a clean and organized way.
- Styling was implemented through **JavaFX CSS**, enabling a modern and consistent visual theme across the application.
- In addition, **JavaFX Animation tools** specifically `TranslateTransition` were used to create smooth interactive effects like the sliding motion in the custom ToggleSwitch component.
- The project also relies on JavaFX Properties (such as `BooleanProperty`, `DoubleProperty`, and `StringProperty`) to ensure real time synchronization between device data and the user interface, allowing the UI to update automatically whenever the device state changes.
- The application follows the **MVC (Model-View-Controller)** architecture to maintain a clean separation between logic, data, and interface.
- The Model layer contains all device classes (such as Camera, AirPurifier, MotionSensor, RGBStrip, and Climate-based devices), each implemented using **Object-Oriented Programming** principles including inheritance, abstraction, and encapsulation.
- The View layer consists of the room interfaces (LivingRoomView, KitchenView, BedroomView, BaseRoomView, and MainView), while the Controller (HomeController) manages the interaction between rooms, devices, and alerts.
- A custom UI component, the ToggleSwitch, was also built using JavaFX shapes and animations to mimic a modern smart-device switch. Although the project includes an `MQTTPublisher interface`, it is used in a simulated manner to mimic IoT communication by printing MQTT-style messages to the console. Finally, data collections like `ObservableList` and `ArrayList` are used to manage rooms, devices, and alert history efficiently.

